

Step 3 — Research Agent — Solution

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Basic Version

Approach 1 — the direct way

```
# agents/research_agent.py
from langchain_core.tools import tool
from langchain_openai import ChatOpenAI
from langchain.agents import create_agent
from pydantic import BaseModel

@tool
def lookup_facts(topic):
    """Look up basic facts about a topic."""
    return [f"{topic} is a growing area of interest.", f"{topic} has multiple vie

class ResearchNotes(BaseModel):
    topic: str
    facts: list

def run_research(topic):
    agent = create_agent(ChatOpenAI(model="gpt-4o-mini"), tools=[lookup_facts])
    result = agent.invoke({"messages": [("user", f"Research this topic using the
    last_message = result["messages"][-1]
    return ResearchNotes(topic=topic, facts=[last_message.content])
```

This works and proves the mechanism — an agent decides to call `lookup_facts`, gets results, and produces something you can wrap into `ResearchNotes`. It's a little sloppy about *how* the facts get extracted (dumping the whole final message into one list item), and it never confirms the tool was actually the source of that final message — both the Intermediate and Advanced versions fix that.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — extracting real, structured facts from the tool message

```
# agents/research_agent.py
from langchain_core.tools import tool
from langchain_openai import ChatOpenAI
from langchain.agents import create_agent
from pydantic import BaseModel

@tool
def lookup_facts(topic: str) -> list[str]:
    """Look up 3-5 basic, factual points about a given topic.
    Always call this before writing research notes – do not rely on your own
    general knowledge instead of this tool's results."""
    return [
        f"{topic} is a growing area of interest.",
        f"{topic} has multiple credible viewpoints worth noting.",
        f"{topic} has measurable trends worth citing with numbers.",
    ]

class ResearchNotes(BaseModel):
    topic: str
    facts: list[str]

def run_research(topic: str) -> ResearchNotes:
    agent = create_agent(ChatOpenAI(model="gpt-4o-mini"), tools=[lookup_facts])
    result = agent.invoke(
        {"messages": [("user", f"Research this topic using the lookup_facts tool:")]
    })

    facts: list[str] = []
    for message in result["messages"]:
        if getattr(message, "name", None) == "lookup_facts":
            # this is the tool's own returned message – the real, structured facts
            facts.extend(message.content if isinstance(message.content, list) else [message.content])

    if not facts:
```

```

        # fall back to the agent's own summary if the tool result wasn't found th
        facts = [result["messages"][-1].content]

    return ResearchNotes(topic=topic, facts=facts)

if __name__ == "__main__":
    for test_topic in ["electric bikes", "remote work", "specialty coffee"]:
        notes = run_research(test_topic)
        print(notes)

```

Difference from Basic: the facts are pulled directly from the tool's own returned message, not the model's final free-text summary — this means `ResearchNotes.facts` holds the real, structured tool output, which is what Step 4's Analysis agent needs to depend on. Full type hints. A fallback path if the tool message isn't found the expected way, instead of the whole function crashing. A real `if __name__ == "__main__":` test block against multiple made-up topics. The fallback, though, is exactly the gap Advanced closes: right now, if the tool truly never got called, this silently falls back to the model's own guess and returns it as if it were verified research.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Approach 1 — verifying the tool actually ran, instead of falling back silently

```
# agents/research_agent.py
from langchain_core.tools import tool
from langchain_openai import ChatOpenAI
from langchain.agents import create_agent
from pydantic import BaseModel

class ToolNotCalledError(Exception):
    pass

@tool
def lookup_facts(topic: str) -> list[str]:
    """Look up 3-5 basic, factual points about a given topic.
    Always call this before writing research notes – do not rely on your own
    general knowledge instead of this tool's results."""
    return [
        f"{topic} is a growing area of interest.",
        f"{topic} has multiple credible viewpoints worth noting.",
        f"{topic} has measurable trends worth citing with numbers.",
    ]

class ResearchNotes(BaseModel):
    topic: str
    facts: list[str]

def run_research(topic: str) -> ResearchNotes:
    agent = create_agent(ChatOpenAI(model="gpt-4o-mini"), tools=[lookup_facts])
    result = agent.invoke(
        {"messages": [("user", f"Research this topic using the lookup_facts tool: " + topic)],
         {"recursion_limit": 8},
    )

    facts: list[str] = []
    for message in result["messages"]:
```

```

        if getattr(message, "name", None) == "lookup_facts":
            facts.extend(message.content if isinstance(message.content, list) else
                          message.content)

    if not facts:
        raise ToolNotCalledError(
            f"Research agent answered on topic '{topic}' without calling lookup_facts "
            f"refusing to return unverified notes."
        )

    return ResearchNotes(topic=topic, facts=facts)

```

Expected output on the normal test topics: the same `ResearchNotes` objects as Intermediate.

Expected behavior if the model ever skips the tool: a raised `ToolNotCalledError` naming the exact topic, instead of a `ResearchNotes` object quietly built from an unverified guess.

Approach 2 — one retry with a more forceful instruction before giving up

```

def run_research(topic: str, retries: int = 1) -> ResearchNotes:
    instruction = f"Research this topic using the lookup_facts tool: {topic}"

    for attempt in range(retries + 1):
        agent = create_agent(ChatOpenAI(model="gpt-4o-mini"), tools=[lookup_facts])
        result = agent.invoke({"messages": [("user", instruction)], {"recursion_limit": 100}})

        facts: list[str] = []
        for message in result["messages"]:
            if getattr(message, "name", None) == "lookup_facts":
                facts.extend(message.content if isinstance(message.content, list) else
                              message.content)

        if facts:
            return ResearchNotes(topic=topic, facts=facts)

        instruction = (
            f"You did not call the lookup_facts tool last time. You MUST call it "
            f"before answering. Research this topic: {topic}"
        )

    raise ToolNotCalledError(f"lookup_facts was never called for topic '{topic}'")

if __name__ == "__main__":
    for test_topic in ["electric bikes", "remote work", "specialty coffee"]:
        print(run_research(test_topic))

```

Difference from Intermediate, and between these 2 Advanced approaches: Intermediate trusts the docstring's "always call this tool" instruction and quietly falls back to an unverified guess if it didn't work — the exact silent-failure shape the `.env` exercise's required-key check and Step 1's empty-notes guard both warned against. Approach 1 fixes that the cheap way: verify, and if verification fails, fail loudly with a specific exception instead of returning something that looks like real research but isn't. Approach 2 goes one step further — instead of giving up immediately, it gives the model one more chance with a more forceful instruction, which in practice recovers a real but occasional failure mode (the model skipping an "optional-sounding" tool call) without ever accepting an unverified result as final.

Which one should you actually write? Approach 1, always, as the floor — a Research agent that can silently skip its own tool and no one would know is a real, quiet bug waiting to reach Step 5's Supervisor and every specialist downstream of it. Approach 2's retry is worth adding once you've actually seen the tool get skipped in testing (rare, but it happens on some topics/phrasings) — it costs one extra API call in the rare failure case and turns "the whole pipeline breaks on this topic" into "it worked on the second try," which is a better failure mode for a real system to have.